

Best practices for Converting MATLAB Code to Fixed Point using Fixed-Point Designer

Fixed-Point Designer helps you design and convert your algorithms to fixed point. Whether you are simply designing fixed-point algorithms in MATLAB or using Fixed-Point Designer in conjunction with one of MathWorks' code generation products, these best practices will help you get from generic MATLAB code to an efficient fixed-point implementation.

This document covers best practices for preparing your code for conversion, efficiently converting your code to fixed point, and optimizing your results. You'll learn tips and tricks for managing bit growth, getting data type proposals based on range information, and optimizing your code to meet your fixed-point constraints.

Overview

1. Prepare Code for Instrumentation, Fixed-Point Conversion, and Acceleration	2
2. Manage Data Types and Control Bit Growth	3
3. Separate Data Type Definitions from Algorithmic Code	4
4. Create Fixed-Point Entries in Types Table.....	5
5. Optimize Data Types	7
6. Optimize your Algorithm	11

1. Prepare Code for Instrumentation, Fixed-Point Conversion, and Acceleration

To ensure a smoother fixed-point conversion process, we recommend that you take the following 3 actions before you start trying to convert your code to fixed point.

a. Create a test file that is separate from your algorithm

A best practice for structuring your code is to separate your core algorithm from other code that you use to test and verify the results. For example, as shown in the following table, you may need to setup some input data to feed into your algorithm, and after you process that data, you may create some plots to verify the results. Since you only need to convert the algorithmic portion to fixed-point, it is more efficient for you to structure your code such that you have a test file, in which you create your inputs, call your algorithm, and plot the results, and one (or more) algorithmic files, in which you do the core processing.

Original code	Best Practice	Modified code
<pre>% TEST INPUT x = randn(100,1); % ALGORITHM y = zeros(size(x)); y(1) = x(1); for n=2:length(x) y(n) = y(n-1) + x(n); end % VERIFY RESULTS yExpected = cumsum(x); plot(y-yExpected) title('Error')</pre>	Issue: Generation of test input and verification of results are intermingled with the algorithm code. Fix: Create a test file that is separate from your algorithm. Put the algorithm in its own function.	<i>Test file.</i> <pre>% TEST INPUT x = randn(100,1); % ALGORITHM y = cumulative_sum(x); % VERIFY RESULTS yExpected = cumsum(x); plot(y-yExpected) title('Error')</pre> <i>Algorithm in its own function.</i> <pre>function y = cumulative_sum(x) y = zeros(size(x)); y(1) = x(1); for n=2:length(x) y(n) = y(n-1) + x(n); end end</pre>

b. Get your algorithmic code instrumentation and acceleration ready

Two key features that Fixed-Point Designer provides are:

1. Ability to instrument your code and provide data type proposals to help you convert your algorithm to fixed point (uses the functions `buildInstrumentedMex`¹ and `showInstrumentationResults`²)
2. Ability to accelerate your fixed-point algorithms by creating a MEX file (uses the `fiaccel`³ function)

Both of these features rely on code generation technology (not a code generation license). Any MATLAB algorithms that you want to instrument using `buildInstrumentedMex` and any fixed-point algorithms that you want to accelerate using `fiaccel` must comply with MATLAB® Coder™ code generation requirements and rules. To view the subset of the

MATLAB language that is supported for code generation, you can examine the tables of functions supported for code generation in the documentation.

To help you identify unsupported functions or constructs in your MATLAB code, there are two tools.

- 1) By adding the `%#codegen` pragma to the top of your MATLAB file, the MATLAB code analyzer will flag functions and constructs that it knows are not available in the subset of the MATLAB language supported for code generation. This advice will appear in real-time as you edit your code in the MATLAB editor.
- 2) For a static report on your code, you can use the Code Generation Readiness tool⁴. To launch this tool for a function named `myFunction1`, type `coder.screener('myFunction1')` at the MATLAB command line. This command will produce a report that identifies calls to functions and the use of data types that are not supported for code generation.

c. Check for fixed-point support for functions used in your algorithmic code

Before you start your fixed-point conversion, it helps to know which functions used in your algorithm aren't supported for fixed point so you can start thinking about how you might replace them or otherwise tweak your implementation to be more optimized for the embedded world. For example, you may need to find (or write your own) replacements for functions like `log2`, `fft`, and `exp`. Other functions like `sin`, `cos`, and `sqrt` may support fixed point, but for better efficiency, you may want to consider an alternative implementation like a lookup table or CORDIC-based algorithm.

For more information on some of these common replacement functions, take a look at the Fixed-Point Designer Featured Examples⁵ in the documentation.

If you can't find a replacement immediately, that's ok too. You can continue converting the rest of your algorithm to fixed point by simply insulating any functions that don't support fixed-point with a cast to double at the input, and a cast back to a fixed-point type at the output.

2. Manage Data Types and Control Bit Growth

The `(:)=` syntax is known as subscripted assignment⁶. When you use this syntax, MATLAB overwrites the value of the left-hand side argument, but retains the existing data type and array size. This is particularly important in keeping fixed-point variables fixed point (as opposed to inadvertently turning them into doubles), and for preventing bit growth⁷ when you want to maintain a particular data type for the output.

Original code	Best Practice	Modified code
<pre>y = 0; for n=1:length(x) y = y + x(n); end</pre>	Issue: "y = y + x(n)" overwrites y with "y + x(n)". This is fine when you are using all double types, but once you start introducing	<pre>y = 0; for n=1:length(x) y(:) = y + x(n); end</pre>

	fixed-point data types in your code, if y is overwritten, it may change data types. Fix: To preserve the original data type of y, assign into y with "y(:)=". This will cast the right-hand-side value into y's pre-defined data type and prevent bit growth.	
--	--	--

3. Separate Data Type Definitions from Algorithmic Code

This method of fixed-point conversion makes it easier for you to compare several different fixed-point implementations, and also allows you to easily retarget your algorithm to a different device.

- When a variable is first defined, use `cast(x,'like',y)` or `zeros(m,n,'like',y)` to cast it to your desired data type.
- Create a table of data type definitions, starting with original data types used in your code. Before converting to fixed point, create a data type table that uses all single data types to find type mismatches and other problems.
- Try running your code connected to each table and look at the results to verify the connection.

Original code	Best Practice	Modified code
<pre>% Algorithm n = 128; y = zeros(size(x));</pre>	Issue: The default data type in MATLAB is double-precision floating-point. Fix: 1) Use "cast(...,'like',...)" and "zeros(...,'like',...)" to programmatically specify types that are defined in a separate table. 2) Create an original types table, usually in a separate function.	<pre>% Algorithm T = mytypes('double'); n = cast(128,'like',T.n); y = zeros(size(x),'like',T.y); % Types Table function T = mytypes(dt) switch(dt) case 'double' T.n = double([]); T.y = double([]); end end</pre>
	3) Add a table of single data types to help verify the connection with your code.	<pre>function T = mytypes(dt) switch(dt) case 'double' T.n = double([]); T.y = double([]); case 'single' T.n = single([]); T.y = single([]); end end</pre>

4. Create Fixed-Point Entries in Types Table

- a. What are your goals for converting to fixed point?
 - i. Are you implementing your algorithm in C or HDL?
 - ii. What are your target constraints? (available word length, fraction length, math modes, math libraries, etc.)
- b. Build and run instrumented to get fixed-point types proposals using the `buildInstrumentedMex` and `showInstrumentationResults` commands. Note that the type proposals are only as good as the test inputs. A simple set of test vectors may not exercise the full range of types, so use the proposals as a guideline for choosing an initial set of fixed-point types, and use your best judgement and experience in adjusting the types. Loop indices are automatically converted to integer types by MATLAB Code when they are only used as index variables, so you do not have to explicitly convert them to fixed-point.

Algorithm Code	Test File
<pre>function [y,z] = myfilter(b,x,z) y = zeros(size(x)); for n=1:length(x) z(:) = [x(n); z(1:end-1)]; y(n) = b * z; end end</pre>	<pre>% Test inputs b = fir1(11,0.25); t = linspace(0,10*pi,256)'; x = sin((pi/16)*t.^2); % Linear chirp z = zeros(size(b'))'; % Build buildInstrumentedMex myfilter ... -args {b,x,z} -histogram % Run [y,z] = myfilter_mex(b,x,z); % Show showInstrumentationResults myfilter_mex ... -defaultDT numerictype(1,16) -proposeFL</pre>

Code Generation Report

Function: myfilter

```
1 function [y,z] = myfilter(b,x,z)
2     y = zeros(size(x));
3     for n=1:length(x)
4         z(:) = [x(n); z(1:end-1)];
5         y(n) = b * z;
6     end
7 end
```

Summary

All Messages (0)

Variables

Order	Variable	Type	Size	Class	Complex	Proposed Signedness	Proposed WL	Proposed FL	Always Whole Number	SimMin	SimMax	Histogram
1	y	Output	256 x 1	double	No	Signed	16	15	No	-0.9514825238809547	0.9831716442570267	
2	z	I/O	12 x 1	double	No	Signed	16	14	No	-0.999998881428792	0.9999956940701684	
3	b	Input	1 x 12	double	No	Signed	16	17	No	-0.004465461051253895	0.24958855452476303	
4	x	Input	256 x 1	double	No	Signed	16	14	No	-0.999998881428792	0.9999956940701684	
5	n	Local	1 x 1	double	No	Signed	16	0	Yes	1	256	

- c.
- d. Create a table of fixed-point data types for areas of your algorithm that you want to convert to fixed-point.

Create the types table using a structure with prototypes for the variables. The proposed types are computed from the simulation runs. A long simulation run with a wide range of expected data will produce better proposals. You can use the proposed types or use your knowledge of the algorithm and implementation constraints from above to improve on the proposals.

Specify the prototype values as empty ([]) since the data types are used, but not the values.

In some cases, it may be more efficient to leave some portions of the code in floating-point. For example, when there is high dynamic range or that portion of the code is sensitive to round-off errors.

Algorithm Code	Types Tables	Test File
<pre>function [y,z] = myfilter(b,x,z,T) y = zeros(size(x),'like',T.y); for n=1:length(x) z(:) = [x(n); z(1:end-1)]; y(n) = b * z; end end</pre>	<pre>function T = mytypes(dt) switch dt case 'double' T.b = double([]); T.x = double([]); T.y = double([]); case 'fixed16' T.b = fi([],true,16,15); T.x = fi([],true,16,15); T.y = fi([],true,16,14); end end end</pre>	<pre>% Test inputs b = fir1(11,0.25); t = linspace(0,10*pi,256)'; x = sin((pi/16)*t.^2); % Linear chirp % Cast inputs T = mytypes('fixed16'); b = cast(b,'like',T.b); x = cast(x,'like',T.x); z = zeros(size(b'),'like',T.x); % Run [y,z] = myfilter(b,x,z,T);</pre>

e. Run with fixed-point types, compare results

Create a test file to validate that the floating-point algorithm works as expected before converting it to fixed point. You can use the same test file to propose fixed-point data types, and to compare fixed-point results to the floating-point baseline after the conversion.

If you encounter functions that aren't supported by fixed-point and can't find a replacement, you can cast the inputs and outputs of the function to double. This will allow you to continue converting your code to fixed-point, and come back when you have a suitable replacement for the unsupported function.

Original code	Best Practice	Modified code
<code>y = 1/exp(x);</code>	Issue: The exp() function is not defined for fixed-point inputs.	<code>y = 1/exp(double(x));</code>

	Fix: Cast the input to double until you have a replacement. In this case, $1/\exp(x)$ is more well-behaved than $\exp(x)$ with respect to fixed-point growth, so you can replace the whole expression with a "one over exp" function, possibly as a lookup table.	
--	---	--

5. Optimize Data Types

- Use scaled doubles to detect potential overflows. Scaled doubles are a hybrid between floating-point and fixed-point numbers. Fixed-Point Designer stores them as doubles with the scaling, sign, and word length information retained. To use scaled doubles, you use data type override (DTO) property.

To...	Use...	Example
Set DTO locally	numerictype 'DataType' property	<pre>>> T.a = fi([], 1, 16, 13,'DataType', 'ScaledDouble'); >> a = cast(pi, 'like', T.a) a = 3.1416 DataTypeMode: Scaled double: binary point scaling Signedness: Signed WordLength: 16 FractionLength: 13</pre>
Set DTO globally	fipref 'DataTypeOverride' property	<pre>>> fipref('DataTypeOverride', 'ScaledDoubles'); >> T.a = fi([], 1, 16, 13); >> a = cast(pi, 'like', T.a) a = 3.1416 DataTypeMode: Scaled double: binary point scaling Signedness: Signed WordLength: 16 FractionLength: 13</pre>

- To fine-tune fixed-point type settings, you can run the `buildInstrumentedMex` function with the `-histogram` flag and run the generated MEX function with your desired test inputs. When you use the `showInstrumentationResults` to display the code generation report, the report will have a Histogram icon, which you can use to

launch the NumericTypeScope⁸ and view the distribution of values observed in your simulation for the selected variable.

Function: `fir_filt_circ_buff_typed_codegen>2` Calls: Select a function call:

```

44 % figure(gcf)
45 %
46 % See also FILTER, FIR_FILT_CIRC_BUFF_FIXED_POINT_CONVERSION_EXAMPLE.
47 %
48 % Copyright 2012 The MathWorks, Inc.
49 %
50 % Use coder.nullcopy only when you are certain that every value of
51 % the variable is overwritten before it
52 y = coder.nullcopy(zeros(size(x),'like'));
53 nx = length(x);
54 nb = length(b);
55 for n=1:nx
56     p(:)=p+1; if p>nb, p(:)=1; end
57     z(p) = x(n);
58     acc = cast(0,'like',T,acc);
59     k = p;
60     for j=1:nb
61         acc(k) = acc + b(j)*z(k);
62         k(k)=k-1;
63     end
64     y(n) = acc;
65 end
66 end

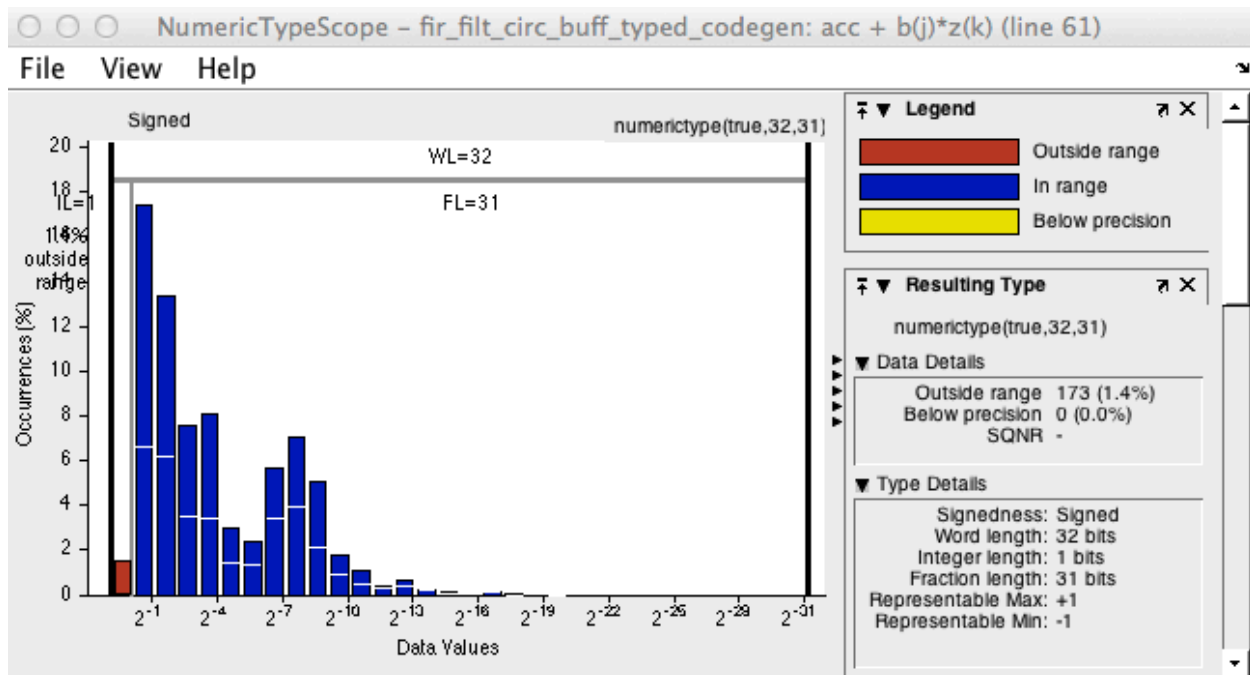
```

5a. When you set DTO to ScaledDoubles, values that would have overflowed are highlighted in red

5b. Quick access to the histogram view showing the distribution of values observed in your simulation

Order	Variable	Type	Size	Class	Complex	DT Mode	Signedness	WL	FL	Percent of Current Range	Always Whole Number	SimMin	SimMax	Histogram
1	y	Output	1 x 1	embedded.f	No	ScaledDouble	Signed	16	14	52	No	-0.9959923266057762	1.035158756226056	
2	z	IO	256 x 1	embedded.f	No	ScaledDouble	Signed	16	14	55	No	-1.0966086573451541	1.0874250377226369	
3	p	IO	1 x 1	embedded.f	No	ScaledDouble	Signed	16	17	100	No	-0.004465461051254	0.249588554524763	
4	b	Input	1 x 12	embedded.f	No	ScaledDouble	Signed	16	14	55	No	-1.0966086573451541	1.0874250377226369	
5	x	Input	256 x 1	embedded.f	No	ScaledDouble	Signed	16	14	55	No	-1.0966086573451541	1.0874250377226369	
6	T	Input	1 x 1	struct	-	-	-	-	-	-	-	-	-	-
7	nx	Local	1 x 1	double	No	-	-	-	-	-	Yes	256	256	
8	nb	Local	1 x 1	double	No	-	-	-	-	-	Yes	12	12	

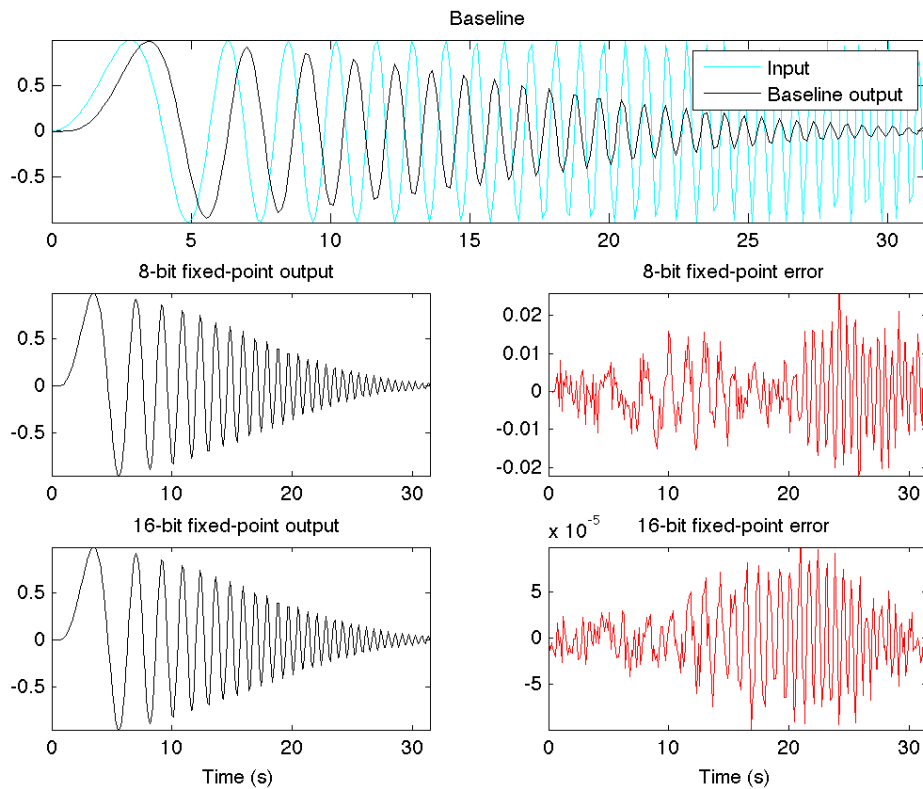
The overflow indicated in red in the Code Generation Report is show in the "outside range" bin in the NumericTypeScope. Launch the NumericTypeScope for an associated variable or expression by clicking on the histogram view icon .



c. Explore design tradeoffs

Once you have your first set of fixed-point data types, you can then add different variations of fixed-point values to your types table. You can modify and iterate to avoid overflows, adjust fraction lengths, and change rounding methods to eliminate bias and so on⁹.

Algorithm Code	Types Tables	Test File
<pre> function [y,z] = myfilter(b,x,z,T) y = zeros(size(x),'like',T.y); for n=1:length(x) z(:) = [x(n); z(1:end-1)]; y(n) = b * z; end end </pre>	<pre> function T = mytypes(dt) switch dt case 'double' T.b = double([]); T.x = double([]); T.y = double([]); case 'fixed8' T.b = fi([],true,8,7); T.x = fi([],true,8,7); T.y = fi([],true,8,6); case 'fixed16' T.b = fi([],true,16,15); T.x = fi([],true,16,15); T.y = fi([],true,16,14); end end </pre>	<pre> function mytest % Test inputs b = fir1(11,0.25); t = linspace(0,10*pi,256)'; x = sin((pi/16)*t.^2); % Linear chirp % Run y0 = entrypoint('double',b,x); y8 = entrypoint('fixed8',b,x); y16 = entrypoint('fixed16',b,x); % Plot subplot(3,1,1);plot(t,x,'c',t,y0,'k'); legend('Input','Baseline output') title('Baseline') subplot(3,2,3);plot(t,y8,'k'); title('8-bit fixed-point output') subplot(3,2,4);plot(t,y0-double(y8),'r'); title('8-bit fixed-point error') subplot(3,2,5);plot(t,y16,'k'); title('16-bit fixed-point output') xlabel('Time (s)') subplot(3,2,6);plot(t,y0-double(y16),'r'); title('16-bit fixed-point error') xlabel('Time (s)') end function [y,z] = entrypoint(dt,b,x) T = mytypes(dt); b = cast(b,'like',T.b); x = cast(x,'like',T.x); z = zeros(size(b'),'like',T.x); [y,z] = myfilter(b,x,z,T); end </pre>



6. Optimize your Algorithm

- Use `fimath` to get natural types for C or HDL

`fimath` properties define the rules for performing arithmetic operations on `fi` objects, including math, rounding, and overflow properties.

You can use the `fimath` `ProductMode` and `SumMode` properties to retain natural data types for C and HDL¹⁰. The `KeepLSB` setting for `ProductMode` and `SumMode` models the behavior of integer operations in the C language, while `KeepMSB` models the behavior of many DSP devices¹¹. Different rounding methods require different amounts of overhead code. Setting the `RoundingMethod` property to `Floor`, which is equivalent to two's complement truncation, provides the most efficient rounding implementation.

Similarly, the standard method for handling overflows is to wrap using modulo arithmetic. Other overflow handling methods create costly logic. It is recommended that you set the `overflowAction` to `Wrap` when possible.

MATLAB Code	Best Practice	Generated C Code
<i>Code being compiled:</i> <pre>function y = adder(a,b) y = a + b; end</pre> <i>With types defined with default fimath settings:</i> <pre>T.a = fi([],1,16,0); T.b = fi([],1,16,0); a = cast(0,'like',T.a); b = cast(0,'like',T.b);</pre>	Issue: Additional code is generated to implement saturation overflow, nearest rounding, and full-precision arithmetic.	<pre>int adder(short a, short b) { int y; int i0; int i1; int i2; int i3; i0 = a; i1 = b; if ((i0 & 65536) != 0) { i2 = i0 -65536; } else { i2 = i0 & 65535; } if ((i1 & 65536) != 0) { i3 = i1 -65536; } else { i3 = i1 & 65535; } i0 = i2 + i3; if ((i0 & 65536) != 0) { y = i0 -65536; } else { y = i0 & 65535; } return y; }</pre>
<i>Code being compiled:</i> <pre>function y = adder(a,b) y = a + b; end</pre> <i>With types defined with fimath settings that match your processor types:</i> <pre>F = fimath(... 'RoundingMethod','Floor', ... 'OverflowAction','Wrap', ... 'ProductMode','KeepLSB', ... 'ProductWordLength',32, ... 'SumMode','KeepLSB', ...</pre>	Fix: Choose fixed-point math settings that match your processor types to make the generated code more efficient.	<pre>int adder(short a, short b) { return a + b; }</pre>

<pre>'SumWordLength',32); T.a = fi([],1,16,0,F); T.b = fi([],1,16,0,F); a = cast(0,'like',T.a); b = cast(0,'like',T.b);</pre>		
---	--	--

- b. Replace builtin functions with more efficient fixed-point implementations.
Some MATLAB built-in functions can be made more efficient for fixed-point implementation. For example, you can replace a built-in function with a Lookup table implementation^{12,13} or a CORDIC implementation^{14,15}, which only requires iterative shift-add operations.
- c. Re-implement division operations where possible
Division is often not fully supported by hardware and can result in slow processing. When your algorithm requires a division, consider replacing it with one of the following options:
 - i. Use bit shifting when the denominator is a power of two (e.g. `bitsra(x,3)` instead of `x/8`)
 - ii. Multiply by the inverse when the denominator is constant (e.g. `x*0.2` instead of `x/5`).
- d. Eliminate floating-point variables
For more efficient code, it is best to eliminate all floating-point variables. The one exception to this is loop indices because they will usually work themselves out to integer types.

Fixed-Point Designer References

¹ Function Reference Page for buildInstrumentedMex:

<http://www.mathworks.com/help/fixedpoint/ref/buildinstrumentedmex.html>

² Function Reference Page for showInstrumentationResults:

<http://www.mathworks.com/help/fixedpoint/ref/showinstrumentationresults.html>

³ Function Reference Page for fiaccel:

<http://www.mathworks.com/help/fixedpoint/ref/fiaccel.html>

⁴ Code Generation Readiness Tool:

<http://www.mathworks.com/help/fixedpoint/ug/code-generation-readiness-tool.html>

⁵ Fixed-Point Designer Featured Examples (MATLAB):

<http://www.mathworks.com/help/fixedpoint/examples/index.html#matlab-examples>

⁶ Fixed-Point Designer Function Reference Page for subsasgn:

<http://www.mathworks.com/help/fixedpoint/ref/subsasgn.html>

⁷ Managing Bit Growth of fi Objects:

http://www.mathworks.com/help/fixedpoint/gs/fixed-point-arithmetic_bt25flf-1.html#bt25flf-3

⁸ Function Reference Page for NumericTypeScope:

<http://www.mathworks.com/help/fixedpoint/ref/numerictypescope.html>

⁹ Demo - Implement an FIR Filter Algorithm for Floating and Fixed-Point Types using Cast and Zeros:

<http://www.mathworks.com/help/fixedpoint/examples/implement-fir-filter-algorithm-for-floating-point-and-fixed-point-types-using-cast-and-zeros.html>

¹⁰ How fi objects compare to C data types: <http://www.mathworks.com/help/fixedpoint/ug/fi-objects-compared-to-c-integer-data-types.html>

¹¹ Fimath product mode and sum mode: <http://www.mathworks.com/help/fixedpoint/ug/using-fimath-productmode-and-summode.html>

¹² Example showing how to normalize data for use in lookup tables:

<http://www.mathworks.com/help/fixedpoint/ug/normalize-data-for-lookup-tables.html>

¹³ Example implementing square root using a lookup table:

<http://www.mathworks.com/help/fixedpoint/examples/implement-fixed-point-square-root-using-lookup-table.html>

¹⁴ Example implementing sine and cosine using CORDIC algorithm:

<http://www.mathworks.com/help/fixedpoint/examples/compute-sine-and-cosine-using-cordic-rotation-kernel.html>

¹⁵ List of CORDIC functions in Fixed-Point Designer Reference:

<http://www.mathworks.com/help/fixedpoint/cordic.html>